

安全审计与加固报告

Web 应用安全审计 与加固实践报告

Flask 用户管理系统 — 从功能开发到安全加固全流程

实验环境: Kali Linux • Python Flask 3.1 • SQLite 3 • Burp Suite CE

靶机地址: <http://192.168.14.129:5000>

项目路径: /root/flask_user_app/

报告日期: 2026-07-07

目录

一、项目概述与功能说明

1.1 项目背景

1.2 系统功能清单

1.3 系统架构图

二、安全审计方法论

2.1 审计范围与攻击面

2.2 漏洞严重程度分级

三、漏洞一：SQL 注入（搜索功能）

3.1 漏洞代码分析

3.2 攻击复现 — OR 注入 + UNION 注入

3.3 Burp Suite 攻击过程记录

3.4 修复方案

四、漏洞二：SQL 注入（注册功能）

4.1 漏洞代码分析

4.2 攻击复现 — 创建任意用户

4.3 修复方案

五、漏洞三：密码明文存储

5.1 漏洞分析与验证

5.2 修复方案与 scrypt 算法详解

六、漏洞四：错误信息泄漏数据库结构

6.1 漏洞分析与攻击复现

6.2 修复方案

七、漏洞五：搜索接口未授权访问

八、漏洞六：跨站请求伪造 CSRF

九、漏洞七：Session Cookie 安全配置缺失

十、漏洞八：输入无长度限制

十一、SQL 注入防护深度解析

11.1 参数化查询工作原理

11.2 各语言参数化写法对比

十二、全部修复方案总结

12.1 修复前后对比总表

12.2 修复代码行数统计

十三、安全开发检查清单

十四、总结与反思

参考资料

一、项目概述与功能说明

1.1 项目背景

本项目是一个基于 Python Flask 框架的用户信息管理平台，分两个阶段完成：

阶段	任务	说明
第一阶段	基础登录功能	实现用户登录、首页信息展示、退出登录
第二阶段（新增）	注册 + 搜索功能	新增用户自主注册、用户搜索功能

本次安全审计在两个阶段全部完成后进行，确保所有功能均经过安全加固。

1.2 系统功能清单

功能	路由	方法	说明	阶段
首页	/	GET	展示当前登录用户的信息	第一阶段
登录	/login	GET/POST	用户名+密码登录，含防爆破	第一阶段
注册	/register	GET/POST	新用户自主注册	新增
搜索	/search	GET	按用户名/邮箱搜索用户	新增
退出	/logout	GET	清除 Session 退出登录	第一阶段

1.3 系统架构图



(登录用)	users.db	Cookie
哈希密码	注册+搜索	

1.4 数据流说明

注册流程：用户填写表单 → POST /register → 验证输入 → INSERT INTO users → 跳转登录页

搜索流程：用户输入关键词 → GET /search?keyword=xxx → SELECT ... LIKE → 表格展示结果

登录流程：用户填写表单 → POST /login → 比对 USERS 字典哈希密码 → 写入 Session → 跳转首页

二、安全审计方法论

2.1 审计范围与攻击面

本次审计覆盖系统全部 5 个路由、2 个数据存储（USERS 字典 + SQLite 数据库）、Session 管理机制。

审计方法

方法	工具	说明
代码审计	人工审查 <code>app.py</code> + 模板文件	逐行检查所有用户输入点
动态测试	Burp Suite + curl	发送恶意 payload 观察响应
黑盒测试	模拟攻击者视角	不查看源码，仅通过接口测试
白盒测试	结合源码分析	定位漏洞代码的确切位置

攻击面总览

攻击面分类	发现数量	严重程度分布
漏洞（SQL 注入）	2 严重 x2	注入类
认证授权类漏洞（CSRF+未授权）	2 中危 x2	数据保护类（明文+信息泄露）
配置类漏洞（Cookie 安全）	1 中危	2 高危+中危
输入校验类（无长度限制）	1	低危
		总计 8
1严重+1高危+5中危+1低危		

2.2 漏洞严重程度分级标准

等级	分值	定义	本次数量
严重 Critical	9.0-10.0	可导致完全的数据泄露或系统接管	2
高危 High	7.0-8.9	可导致敏感数据泄露	1
中危 Medium	4.0-6.9	在特定条件下可被利用	4
低危 Low	1.0-3.9	单独利用难度大或影响有限	1

三、漏洞一：SQL 注入（搜索功能）

漏洞信息

项目	内容
漏洞类型	SQL 注入 (SQL Injection)
危害等级	严重 (Critical) — CVSS 9.8
CWE 编号	CWE-89: SQL Injection
影响范围	搜索接口 <code>GET /search?keyword=xxx</code> （今日新增功能）
发现方式	白盒代码审计 + 黑盒动态测试

3.1 漏洞代码分析

问题代码（`app.py`，修复前）：

代码

```
@app.route("/search")
def search():
    keyword = request.args.get("keyword", "") # 用户可控输入
    sql = f"SELECT id, username, email, phone FROM users \
        WHERE username LIKE '{keyword}%' OR email LIKE '{keyword}%'"
    #
    # f-string 直接拼接      两处都有注入点
    c.execute(sql)
```

根本原因分析（逐层拆解）

第1层：使用了 f-string（格式化字符串） `f"WHERE username LIKE '{keyword}%'"` 这里的 `{keyword}` 会被 Python 替换为用户输入的值 第2层：用户输入未经过滤 输入 `' OR '1'='1'` 中的单引号 `'` 被直接放入 SQL 第3层：单引号闭合了 SQL 字符串 `LIKE '% OR '1'='1%'` ↑ 这里 `'` 闭合了前面 `LIKE` 后的字符串 第4层：`OR` 追加了永真条件 `... OR '1'='1%'` ↑ 永真条件，使 `WHERE` 子句永远为 `true`

3.2 攻击复现 — POC 1: OR 万能注入

攻击目标：获取 `users` 表中所有用户的全部数据。

攻击命令：

代码

```
# 注入 payload: ' OR '1'='1
# URL 编码后: %27%20OR%20%271%27%3D%271

curl "http://127.0.0.1:5000/search?keyword=%27%20OR%20%271%27%3D%271"
```

攻击结果（修复前）：

代码

```
数据库返回全部记录：
ID: 1, username: admin, email: admin@example.com
ID: 2, username: alice, email: alice@example.com
ID: 3, username: hacker, email: h@x.com ← 之前注册注入创建的
```

攻击复现 — POC 2: UNION 注入

攻击目标：在正常搜索结果中注入自定义伪造数据。

代码

```
# UNION SELECT 注入 — 将自定义结果合并到查询中
curl "http://127.0.0.1:5000/search?keyword=%27%20UNION%20SELECT%201,'inj','inj@x.com','138'--"

# 攻击结果：
# 正常数据: ID:1 admin, ID:2 alice
# 伪造数据: ID:1, username: inj, email: inj@x.com, phone: 138
```

攻击原理：UNION 是 SQL 中的集合操作，用于合并两个 SELECT 查询的结果。攻击者通过 UNION 将自己的 SELECT 结果附加到正常结果后面。关键约束：UNION 两边的列数必须相等。搜索查询有 4 列 (id, username, email, phone)，所以 UNION SELECT 也必须提供 4 个值。

3.3 Burp Suite 攻击过程记录

Burp Suite 配置：
Proxy → Proxy Settings → Add
Bind to port: 8080
Bind to address: 127.0.0.1
Support Invisible Proxying: ON

拦截后经 Repeater 修改参数测试：

测试1: keyword=admin' OR '1'='1

响应长度: ~3000 bytes (比正常长50%) → 注入成功

测试2: keyword=' UNION SELECT 1,2,3,4--

响应中出现: 1, 2, 3, 4 → UNION 注入成功, 列数为 4

3.4 修复方案

修复思路：使用参数化查询（Parameterized Query）替代 f-string 拼接，同时添加登录认证。

```
@app.route("/search")
@login_required # 新增：必须登录才能搜索
def search():
    keyword = request.args.get("keyword", "")
    if keyword:
        if len(keyword) > 100:
            keyword = keyword[:100] # 新增：长度限制

        # 修复：使用 ? 占位符替代 f-string
        sql = "SELECT id, username, email, phone FROM users \
            WHERE username LIKE ? OR email LIKE ?"
        safe_keyword = f"%{keyword}%"
        c.execute(sql, (safe_keyword, safe_keyword))
        # 参数单独传入，不会变成 SQL 代码
```

修复原理对比：

修复前 (f-string) : "WHERE username LIKE '%{keyword}%" → 用户输入 ' OR '1'='1 → SQL: WHERE username LIKE '%" OR '1'='1%' → 注入成功

修复后 (参数化) : "WHERE username LIKE ?" → 用户输入 ' OR '1'='1 → 传给 DB 的参数值为 '%" OR '1'='1%' → 数据库查找包含该文本的用户 → 注入失败

四、漏洞二：SQL 注入（注册功能）

漏洞信息

项目	内容
漏洞类型	SQL 注入 (SQL Injection)
危害等级	严重 (Critical) — CVSS 9.8
影响范围	注册接口 POST /register （今日新增功能）
攻击入口	用户名、密码、邮箱、手机号四个字段均可注入

4.1 漏洞代码分析

问题代码（`app.py`，修复前）：

```
@app.route("/register", methods=["GET", "POST"])
def register():
    ...
    username = request.form.get("username", "")
    # f-string 拼接 — 4个字段全部可注入!
    sql = f"INSERT INTO users (username, password, email, phone) \
        VALUES ('{username}', '{password}', '{email}', '{phone}')"
    c.execute(sql)
```

代码

4.2 攻击复现 — POC 3：注册注入创建任意用户

攻击原理：在用户名中注入闭合括号 + VALUES 语句，创建攻击者控制的用户。

```
# 注入 payload: hacker', 'pass', 'h@x.com', '123')--
curl -X POST http://127.0.0.1:5000/register \
  -d "username=hacker', 'pass', 'h@x.com', '123')--" \
  -d "password=随便填"

# 验证注入是否成功
sqlite3 data/users.db "SELECT id, username, password FROM users;"
# 输出:
# 1|admin|admin123
# 2|alice|alice2025
# 3|hacker|pass      ← 攻击者创建的非法用户!
```

代码

更为严重的攻击场景：

场景1：删除表 — `username = ' ; DROP TABLE users; --`

场景2：修改管理员密码 — `username = ' ; UPDATE users SET password='hacked' WHERE username='admin'; --`

4.3 修复方案

```
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        # 防护1: 输入验证 + 长度限制
        valid, msg = validate_input(username, 30)
        if not valid:
            return render_template("register.html", error=msg)

        # 防护2: 密码哈希存储
        password_hash = generate_password_hash(password)

        # 防护3: 参数化查询 — 核心防御!
        c.execute(
            "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)",
            (username, password_hash, email, phone),
        )
        conn.commit()
```

代码

五、漏洞三：密码明文存储

漏洞信息

项目	内容
漏洞类型	CWE-312: Cleartext Storage of Sensitive Information
危害等级	高危 (High) — CVSS 7.5
影响范围	SQLite 数据库文件 <code>data/users.db</code>

5.1 漏洞分析与验证

直接验证：

```
sqlite3 data/users.db "SELECT id, username, password FROM users;"
1|admin|admin123      ← 明文可见！
2|alice|alice2025    ← 明文可见！
```

代码

风险场景

风险	说明
SQL 注入导致泄露	攻击者通过 OR 注入可以读取 password 字段
数据库文件泄露	如果服务器被入侵， <code>data/users.db</code> 可被直接下载
密码复用攻击	用户在多个平台使用相同密码，一个泄露 = 全部沦陷
内部人员威胁	运维或数据库管理员可直接查看明文密码

5.2 修复方案与 script 算法详解

```
from werkzeug.security import generate_password_hash

# 修复：使用哈希存储代替明文
("admin", generate_password_hash("admin123"), ...)
```

代码

Werkzeug scrypt 哈希算法详解

```
generate_password_hash("admin123")
```

生成的哈希值结构：

```
scrypt:32768:8:1$abcdefghijklmnopqrstu
```

```
vwxyz1234567890... 算法 迭代 r p
```

Salt (16字节) Hash (32字节) 名称 成本 参数 参数 随机生成，每次不同 算法安全性对比 (8位密码穷举时间)： MD5： 约 1

小时 SHA256： 约 5 小时 scrypt： 约 4 年

修复后验证：

```
sqlite3 data/users.db "SELECT id, username, password FROM users;"
```

输出： 1|admin|scrypt:32768:8:1\$aBcDeFgHiJkLmNoP\$aBcDeFgHiJkLmNoPqRStUvWxYz...

即使数据库泄露，攻击者也无法从哈希值还原出原始密码。

六、漏洞四：错误信息泄漏数据库结构

漏洞信息

项目	内容
漏洞类型	CWE-209: Information Exposure Through an Error Message
危害等级	中危 (Medium)
影响范围	注册接口 <code>POST /register</code>

6.1 漏洞分析与攻击复现

问题代码（修复前）：

```
except Exception as e:
    error = f"注册失败: {e}"    # 直接将异常对象展示给用户
```

代码

攻击复现：

```
curl -X POST http://127.0.0.1:5000/register \
-d "username=admin&password=test123"

# 服务器返回:
"注册失败: UNIQUE constraint failed: users.username"
```

代码

从错误信息中可推断的情报

1. 数据库类型: SQLite (错误信息是 SQLite 特有格式) | 2. 表结构: users 表存在 | 3. 字段约束: username 字段有 UNIQUE 约束 | 4. 有效用户: admin (因为触发了冲突) | → 攻击者可利用此信息进行用户名枚举

6.2 修复方案

```
# 修复前：直接暴露异常信息
except Exception as e:
    error = f"注册失败: {e}"

# 修复后：分级处理，统一提示
except sqlite3.IntegrityError:
```

代码

```
        error = "该用户名已被注册，请换一个。"  
except Exception:  
    error = "注册失败，请稍后重试。"  
    # ↑ 统一提示，不暴露具体原因
```

七、漏洞五：搜索接口未授权访问

漏洞信息

项目	内容
漏洞类型	CWE-862: Missing Authorization — 未授权访问
危害等级	中危 (Medium)
影响范围	搜索接口 GET /search (新增功能)

问题分析：搜索接口没有 `@login_required` 装饰器，未登录用户也可触发 SQL 查询。

✖ 修复前

```
@app.route("/search")
def search(): # 未检查登录
```

✔ 修复后

```
@app.route("/search")
@login_required
def search():
```

```
# 登录验证装饰器
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "username" not in session:
            return redirect("/login") # 未登录 → 跳转登录页
        return f(*args, **kwargs)
    return decorated
```

代码

八、漏洞六：跨站请求伪造（CSRF）

漏洞信息

项目	内容
漏洞类型	CWE-352: Cross-Site Request Forgery
危害等级	中危 (Medium)
影响范围	登录 <code>POST /login</code> 、注册 <code>POST /register</code>

攻击场景模拟

假设受害者已登录系统 (Session Cookie 在浏览器中) 。攻击者构造的恶意页面：

```
<form action="http://192.168.14.129:5000/register" method="POST" style="display:none">
<input name="username" value="attacker">
<input name="password" value="hacked">
</form>
<script>document.forms[0].submit()</script>
```

当受害者访问该页面 → 浏览器自动提交表单 + 附加 Cookie → 攻击者账号被创建

修复方案

代码

```
import secrets

@app.before_request
def ensure_csrf():
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_hex(16)

def validate_csrf():
    token = request.form.get("csrf_token", "")
    return token and token == session.get("csrf_token", "")
```

前端表单添加 `<input type="hidden" name="csrf_token" value="{{ csrf_token }}">` 。

九、漏洞七：Session Cookie 安全配置缺失

漏洞信息

项目	内容
漏洞类型	CWE-1004: Sensitive Cookie Without HttpOnly
危害等级	中危 (Medium)
影响范围	全部需要登录的页面

各 Cookie 安全属性的作用

属性	作用	不加的风险
HttpOnly	禁止 JavaScript 读取 Cookie	XSS 攻击者可窃取 Session
SameSite=Lax	仅同站请求发送 Cookie	CSRF 攻击可跨站利用
Secure	仅 HTTPS 下发送 Cookie	HTTP 下 Cookie 明文传输

修复方案

```
app.config.update(  
    SESSION_COOKIE_HTTPONLY=True,      # 禁止 JS 读取  
    SESSION_COOKIE_SAMESITE="Lax",     # 防护 CSRF  
    SESSION_COOKIE_SECURE=False,       # 内网 HTTP 暂不启用  
)
```

代码

修复后验证：

修复前：Set-Cookie: session=eyJ...; Path=/

修复后：Set-Cookie: session=eyJ...; HttpOnly; Path=/; SameSite=Lax

十、漏洞八：输入无长度限制

漏洞信息

项目	内容
漏洞类型	输入验证不足（Input Validation）
危害等级	低危（Low）
影响范围	登录、注册、搜索全部输入点

修复方案

代码

```
def validate_input(data, max_len=50):
    """统一输入验证函数"""
    if not data or not data.strip():
        return False, "输入不能为空"
    if len(data) > max_len:
        return False, f"输入过长（最多{max_len}字符）"
    return True, ""

# 前端同样限制，双重保护：
# <input type="text" name="username" maxLength="30">
```

十一、SQL 注入防护深度解析

11.1 参数化查询的工作原理

注入成功的三个条件

- 条件1: 程序使用字符串拼接构建 SQL (如 f-string / %s / +)
- 条件2: 用户输入包含 SQL 元字符 (主要是单引号 ')
- 条件3: 拼接后的 SQL 语法合法 (攻击者构造出语法正确的语句)

参数化查询执行流程

参数化查询执行流程 步骤1: 应用发送 SQL 模板给数据库 "SELECT * FROM users WHERE username LIKE ?" ↑ 占位符 ? 步骤2: 数据库解析 SQL 模板, 生成查询执行计划 此时 SQL 结构已固定! 步骤3: 应用单独发送参数值 参数值: "%' OR '1'='1%" 步骤4: 数据库将参数填入执行计划 但不会重新解析为 SQL 代码! LIKE 匹配文本: "%' OR '1'='1%" ↑ 这是要找的字符串内容, 不是 SQL 指令!

f-string 拼接 vs 参数化查询对比

步骤	f-string 拼接	参数化查询
第1步	<code>f"WHERE name = '{input}'"</code> — 输入和代码混在一起	<code>"WHERE name = ?"</code> — 模板和数据分离
第2步	输入 <code>' OR '1'='1</code> 成为 SQL 代码	输入被当作参数, 不是代码
第3步	生成 <code>WHERE name = ' OR '1'='1'</code>	数据库说: 这是参数值
结果	注入成功	注入失败

11.2 各编程语言的参数化查询写法

语言	驱动	参数化（安全）	危险写法
Python	sqlite3	<code>c.execute("SELECT ?", (val,))</code>	<code>f"SELECT '{val}'"</code>
Python	MySQLdb	<code>c.execute("SELECT %s", (val,))</code>	<code>"SELECT '%s' % val"</code>
Python	psycopg2	<code>c.execute("SELECT %s", (val,))</code>	<code>f"SELECT '{val}'"</code>
PHP	PDO	<code>\$stmt->execute([':v'=>\$val])</code>	<code>"SELECT '\$val'"</code>
Java	JDBC	<code>pstmt.setString(1, val)</code>	<code>"SELECT '" + val + "'"</code>
Node.js	mysql2	<code>conn.query("SELECT ?", [val])</code>	<code>\`SELECT '\${val}'\`</code>

多层注入防护体系

SQL 注入防护体系 第一层（最有效）：参数化查询 / 预编译语句 原理：数据与代码在数据库层面分离 效果：几乎 100% 防御 SQL 注入 第二层（辅助）：输入验证与过滤 角色：作为第一层的补充，而非替代 第三层（辅助）：最小权限原则 效果：即使注入成功，也能限制损害范围 第四层（辅助）：Web 应用防火墙 WAF 角色：最后一道防线

十二、全部修复方案总结

12.1 修复前后对比总表

#	漏洞名称	严重	修复前	修复后
1	SQL 注入（搜索）	严重	f"SELECT ... LIKE '%{k}%'"	"... LIKE ?" 参数绑定
2	SQL 注入（注册）	严重	f"VALUES ('{v}')"	"VALUES (?)" 参数绑定
3	密码明文存储	高危	INSERT ('admin123')	generate_password_hash()
4	错误信息泄露	中危	f"注册失败: {e}"	统一提示
5	未授权访问	中危	搜索接口无认证	@login_required
6	CSRF 无防护	中危	表单无 Token	secrets.token_hex(16)
7	Cookie 安全缺失	中危	仅 Path=/ 	HttpOnly; SameSite=Lax
8	输入无限制	低危	无校验	validate_input()

12.2 修复代码行数统计

文件	修复前	修复后	新增代码
app.py	84	~190	~106 行（安全加固代码）
login.html	21	24	3 行（CSRF Token）
register.html	31	34	3 行
index.html	63	64	搜索功能（新增）
style.css	196	220+	搜索表格样式

8 发现漏洞	8 已修复	~110 新增代码行	6 安全机制
-----------	----------	---------------	-----------

十三、安全开发检查清单

SQL 注入防护检查清单

- ☒ 所有 SQL 操作是否使用了参数化查询（? / %s / \$1 占位符）？
- ☒ 是否完全禁止了 f-string / % 拼接 / + 拼接 SQL？
- ☒ 用户输入在 LIKE 子句中是否仍使用占位符？
- ☐ ORDER BY、表名等无法参数化的部分做了白名单校验？
- ☒ 错误信息是否没有暴露 SQL 语法或数据库结构？

通用 Web 安全检查清单

- ☒ 所有敏感接口是否有登录验证？
- ☒ 密码是否使用强哈希算法（scrypt / bcrypt / argon2）？
- ☒ 登录是否有防爆破机制？
- ☒ Session 是否配置了 HttpOnly + SameSite 属性？
- ☒ Session Secret Key 是否随机生成？
- ☐ 生产环境是否强制 HTTPS？
- ☒ 所有用户输入是否有长度和格式校验？
- ☒ 是否使用 CSRF Token 保护 POST 请求？
- ☒ Debug 模式是否已关闭？

十四、总结与反思

实验核心收获

- 原则 1

不信任：永远不要信任用户的任何输入
- 原则 2

不拼接：永远不要用字符串拼接构建 SQL
- 原则 3

不泄露：永远不要在错误信息中暴露内部细节

SQL 注入攻防的核心逻辑

攻击者：闭合引号 → 注入指令 → 注释掉剩余代码
防御者：参数化查询 → 数据代码分离 → 纵深防御

本次实践关键数据

指标	数据
发现的漏洞总数	8 个
已修复的漏洞数	8 个（修复率 100%）
新增安全代码行数	~110 行
使用的安全机制	6 种（参数化查询+哈希+CSRF+Cookie+输入验证+认证）
注入成功到注入失败	只需将 f-string 改为 ? 占位符

后续学习方向

方向	说明	难度
XSS 跨站脚本	在页面中注入 JavaScript	进阶
SSRF	利用服务端发起内网请求	进阶
文件上传漏洞	上传 Webshell 获取服务器权限	高阶
NoSQL 注入	MongoDB 等非关系型数据库注入	高阶
SSTI 模板注入	Jinja2 等模板引擎的注入攻击	高阶

安全不是一种功能，而是一种属性。

它不是在开发完成后"加"上去的，而是在每一行代码中"设计"出来的。

通过本次实验，从一个简单的 Flask 登录系统出发，经历了功能扩展（注册+搜索）、漏洞挖掘（8个漏洞）、安全加固（6种防护机制）的完整流程，建立了从"攻击者视角"思考问题、从"防御者视角"编码实践的双重视角。

参考资料

#	资料名称	来源
1	OWASP Top 10 — 2021	https://owasp.org/www-project-top-ten/
2	OWASP SQL Injection Prevention Cheat Sheet	cheatsheetseries.owasp.org
3	OWASP CSRF Prevention Cheat Sheet	cheatsheetseries.owasp.org
4	OWASP Input Validation Cheat Sheet	cheatsheetseries.owasp.org
5	PortSwigger — SQL Injection	portswigger.net/web-security/sql-injection
6	PortSwigger — CSRF	portswigger.net/web-security/csrf
7	Werkzeug 密码哈希文档	werkzeug.palletsprojects.com
8	Flask Session Cookie 配置	flask.palletsprojects.com
9	SQLite 参数化查询	sqlite.org/c3ref/bind_blob.html
10	CWE-89: SQL Injection	cwe.mitre.org
11	CWE-312: Cleartext Storage	cwe.mitre.org
12	CWE-352: Cross-Site Request Forgery	cwe.mitre.org

报告人：_____ 指导教师：_____

报告日期：2026-07-07 | 项目地址：<http://192.168.14.129:5000>

源码路径：[/root/flask_user_app/](#) | 源文件：[Web安全审计与加固报告.md](#)

— 报告结束 —